Logging and Observability

Contents

Logging and Observability **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 3
- Related . . . 3

Logging and Observability

Scope

Application logging, the mandatory client-action audit log, and basic telemetry.

Official requirements

Subject V.6, mandatory and graded:

> Every action in the mobile application must create a log on the backend, containing the platform (Android, iOS…), the device (iPhone 6G, iPad Air, Samsung Edge…) and the application version.

Project policy

The audit log is not the application log

Two separate concerns, two separate stores:

	Audit log	Application log
Purpose	satisfy subject V.6, record what a user did	diagnose the system
Store	database table, queryable	structured log sink
Retention	kept, it is evidence	rotated
Written by	a pipeline behaviour on every command	anywhere

Audit record fields: `userId`, `action`, `targetType`, `targetId`, `platform`, `device`, `appVersion`, `occurredAtUtc`, `correlationId`, outcome.

`platform`, `device` and `appVersion` arrive as `X-Client-Platform`, `X-Client-Device`, `X-Client-App-Version` headers. Middleware validates them and puts them in the request scope. A mutating request without them is rejected with `400` — a silent default would make the audit log useless and the requirement unmet.

Both transports must be covered. REST commands are audited by the pipeline behaviour. SignalR `JoinEvent`, `LeaveEvent`, `JoinPlaylist` and `LeavePlaylist` are audited by a **hub filter** writing the same record — a hub method does not pass through the REST pipeline, so without the filter those actions fall out of the audit and subject V.6 is not met. Client context is captured at the **handshake**, because a hub invocation carries no HTTP headers; a handshake without it is rejected. Connect and disconnect are deliberately **not** audited: mobile reconnect noise would bury deliberate user actions. They stay in the application log. Decision: `brain/decisions/ADR-0019-hub-action-auditing.md`.

Application logging

- **Serilog** writes structured JSON to stdout; **OpenTelemetry** carries traces and metrics, with the exporter configuration-driven and **off by default** (`brain/decisions/ADR-0022-observability-stack.md`).
- Structured logging only (`ILogger<T>` with message templates). Never string-interpolate values into the message — `_log.LogInformation("Vote cast for {TrackId}", trackId)`.
- The traces exist to answer subject V.7's "name the bottleneck with evidence". If the load report names a bottleneck without a trace behind it, this stack was decoration — see the load standard.
- Levels: `Error` for a failure needing action, `Warning` for a handled anomaly, `Information` for a business milestone, `Debug` for development detail. `Information` is the production floor.
- Every request carries a `correlationId` / `traceId`, returned in the error payload, so a user report maps to a log line.
- Log at the boundary: one entry in and one out per request, plus explicit domain events. Do not log inside loops.

Never logged

Passwords, tokens, refresh tokens, API keys, `Authorization` headers, full request bodies of credential endpoints, any secret, and **voter coordinates** submitted for a location licence check (threat T-21). See `../../40-security/secrets-management/STANDARD.md`.

Security events worth logging

Failed login, lockout, refresh-token reuse detection, authorisation denial, rate-limit trip, YouTube quota exhaustion. These are the ones a defence question will reach for.

Health and metrics

- `/health` liveness and `/health/ready` readiness endpoints, unauthenticated but minimal.
- Track request rate, error rate, latency percentiles, and YouTube call count. These feed the load analysis in `../load-testing/STANDARD.md`.
- During a load run, switch the OpenTelemetry exporter on and capture one trace showing the EF Core span, the outbound `HttpClient` span and total request time together. That single trace is the evidence the bottleneck claim rests on.

Checklist

- ☐ Audit table exists and is written by a behaviour, not by hand per handler
- ☐ Client context headers validated and required on mutations
- ☐ Hub filter registered; join and leave produce the same audit record
- ☐ Handshake without client context is rejected
- ☐ Structured templates everywhere, no interpolated messages
- ☐ Correlation id in every request and every error response
- ☐ No secret reachable in any sink
- ☐ Health endpoints present

References

- .NET logging — https://learn.microsoft.com/dotnet/core/extensions/logging
- OpenTelemetry .NET — https://opentelemetry.io/docs/languages/net/
- OWASP Logging Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

Related

`../../40-security/application-security/STANDARD.md` · `../load-testing/STANDARD.md`